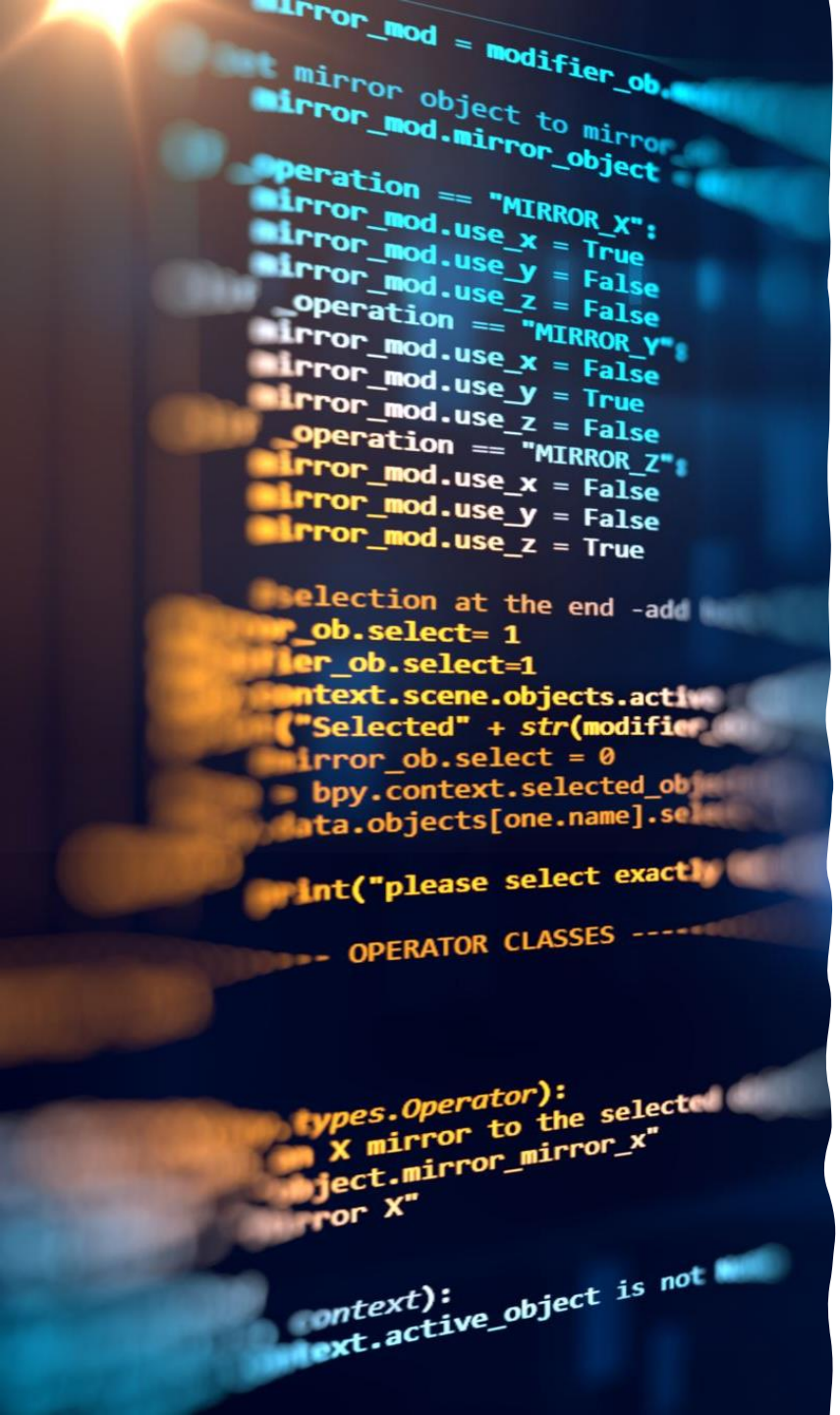


Pengenalan Backend Programming

Tim Dosen PSS

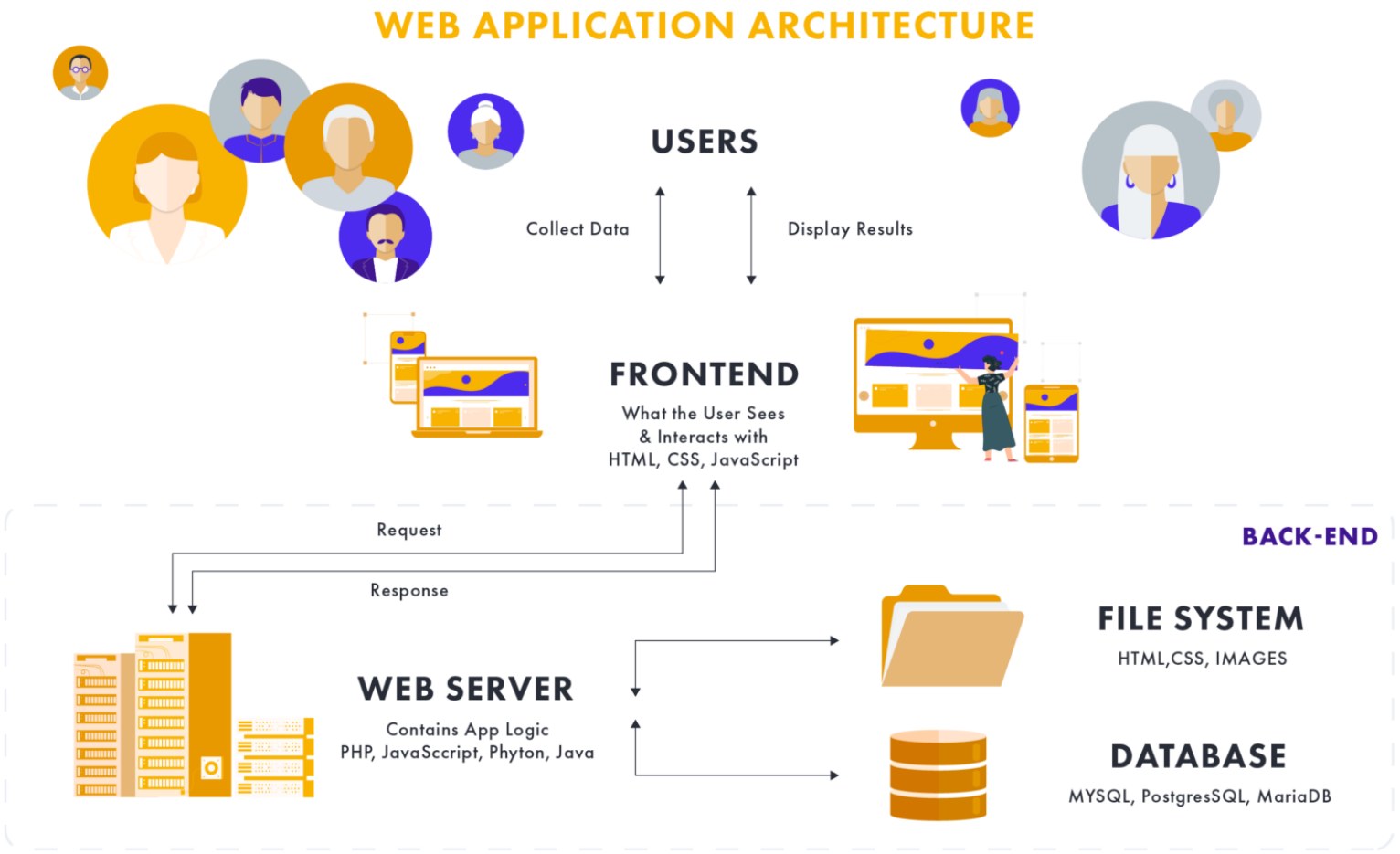
Struktur Perkuliahan

- Waktu Kuliah: 3 SKS (teori)
  - Persiapan membawa laptop & colokan untuk praktik di kelas
- Persentase Nilai:
  - UTS : 35%
  - UAS : 35%
  - Tugas : 30%
- Proyek akhir membuat backend aplikasi yang sudah *containerized* dan *tested*



Apa itu Backend Programming

Backend Programming merujuk pada pengembangan bagian belakang dari sebuah aplikasi web atau perangkat lunak. Ini melibatkan pemrograman server-side, yaitu kode yang berjalan di server dan bertanggung jawab untuk memproses data, melakukan logika bisnis, dan berinteraksi dengan database.



Kenapa Fokus ke Backend?

Fokus ke satu topik pengembangan:	• Pembahasan yang terlalu luas, mengurangi kedalaman materi pada bidang tertentu. Sehingga perlu matkul khusus yang membahas backend secara mendalam.
Memahami Arsitektur Aplikasi dengan Lebih Baik:	• Memahami bagaimana backend bekerja akan membantu memahami keseluruhan arsitektur aplikasi dan membuat kita menjadi pengembang yang lebih lengkap.
Menghasilkan Aplikasi yang Lebih Kuat dan Lebih Fleksibel:	• Kita dapat membangun aplikasi yang lebih kompleks dan fleksibel dengan mengimplementasikan logika bisnis yang rumit dan mengelola data lebih efisien.
Peningkatan Peluang Karier:	• Banyak perusahaan teknologi membutuhkan pengembang backend untuk membangun dan mengelola infrastruktur.

Apa Saja yang Harus Dipelajari?

Bahasa Pemrograman:

- Perlu mempelajari bahasa pemrograman yang cocok untuk backend development. Contoh: JavaScript (Node.js), Python, Ruby, Java, PHP, dll.

Framework dan Library:

- Untuk mempercepat pengembangan, kita menggunakan framework dan library yang tersedia. Contoh: Express.js untuk Node.js, Django untuk Python, Spring untuk Java, dan Laravel untuk PHP.

Database:

- Perlu memahami dasar-dasar database dan cara berinteraksi menggunakan SQL atau menggunakan ORM (Object-Relational Mapping) untuk menyederhanakan akses database.

HTTP dan RESTful API:

- Memahami protokol HTTP dan bagaimana membangun RESTful API (Application Programming Interface) adalah keterampilan penting untuk komunikasi antara aplikasi frontend dan backend.

Apa Saja yang Harus Dipelajari ? (2)

Keamanan:

- Memahami praktik keamanan seperti enkripsi data, perlindungan terhadap serangan siber, dan otentikasi pengguna untuk melindungi aplikasi Anda dari ancaman keamanan.

Manajemen Sesi dan Otentikasi:

- Memahami bagaimana mengelola sesi pengguna dan otentikasi mereka adalah kunci untuk membangun aplikasi web yang aman dan dapat diandalkan.

Deployment dan Skalabilitas:

- Memahami cara deploy aplikasi server dan mengelola skalabilitasnya untuk menangani lalu lintas yang meningkat.

Backend vs Full Stack

Backend Programming:

Fokus Utama:

- Backend programming fokus pada pengembangan bagian belakang (server-side) dari sebuah aplikasi. Ini meliputi logika bisnis, pemrosesan data, interaksi dengan database, dan menyediakan layanan API untuk berkomunikasi dengan frontend atau klien.

Tugas Khusus:

- Pengembang backend biasanya bertanggung jawab untuk mengelola server, konfigurasi database, mengelola cache, dan menangani logika yang terkait dengan server.

Bahasa dan Framework:

- Seorang pengembang backend memilih bahasa pemrograman tertentu dan mungkin menggunakan beberapa framework yang dikhususkan untuk backend development, seperti Express.js untuk Node.js, Django untuk Python, atau Spring untuk Java.

Fullstack Programming:

Gabungan Frontend dan Backend:

- Fullstack programming mencakup pengembangan baik di sisi klien (frontend) maupun di sisi server (backend). Ini berarti seorang pengembang fullstack dapat bekerja di semua aspek pengembangan aplikasi, mulai dari tampilan pengguna hingga logika bisnis dan database.

Kemampuan Lintas Disiplin:

- Seorang fullstack developer memiliki pengetahuan tentang banyak aspek pengembangan, termasuk desain UI/UX, pengembangan frontend, pengembangan backend, manajemen database, dan mungkin bahkan administrasi server.

Fleksibilitas:

- Fullstack developers memiliki fleksibilitas yang lebih besar dalam proyek-proyek kecil atau dalam tim yang lebih kecil, karena mereka dapat berkontribusi pada semua bagian dari aplikasi.

Perbedaan Utama:

Backend programming terfokus hanya pada pengembangan sisi server, sedangkan fullstack programming mencakup kedua sisi, klien dan server.

Seorang fullstack developer memiliki pengetahuan lintas disiplin yang lebih luas daripada pengembang backend biasa.

Pengembang backend mungkin lebih spesialis dalam domain tertentu, sementara pengembang fullstack memiliki pengetahuan yang lebih umum.

Silabus Backend UDINUS

Architectural Pattern

- Arsitektur Monolitik
- Service Oriented Architecture (SOA)
- Arsitektur Microservices
- Arsitektur Serverless

Containerization

Database Modelling

- Object Relational Mapping
- ACID Transaction
- N+1 Problems

NoSQL Database

- Key Value Database
- Document Oriented Database
- Graph Database

API Specification

- OPEN API Specification
- Authentication

Message Brokers

- Message Queue
- Event Streaming

Architectural Pattern

Arsitektural pattern (pola arsitektur) adalah pendekatan atau kerangka kerja yang digunakan dalam pengembangan perangkat lunak untuk merancang struktur keseluruhan dari sebuah sistem.

Pola arsitektur memberikan panduan tentang bagaimana komponen-komponen perangkat lunak harus diorganisir, berinteraksi, dan berkomunikasi satu sama lain.

Tujuan utama dari pola arsitektur membantu para pengembang dalam mengatasi masalah desain yang umum ditemui, meningkatkan kualitas sistem, dan mempercepat proses pengembangan.

Contoh pola arsitektur:

Model-View-Controller (MVC)	Client-Server	Layered Architecture	Microservices	Event-Driven Architecture	Service-Oriented Architecture

Containerization

Metode untuk mengemas, distribusi, dan menjalankan aplikasi secara konsisten di lingkungan komputasi yang terisolasi.

Kontainer menyertakan semua elemen yang diperlukan untuk menjalankan aplikasi, seperti kode, runtime, sistem operasi, pustaka, dan pengaturan lingkungan.

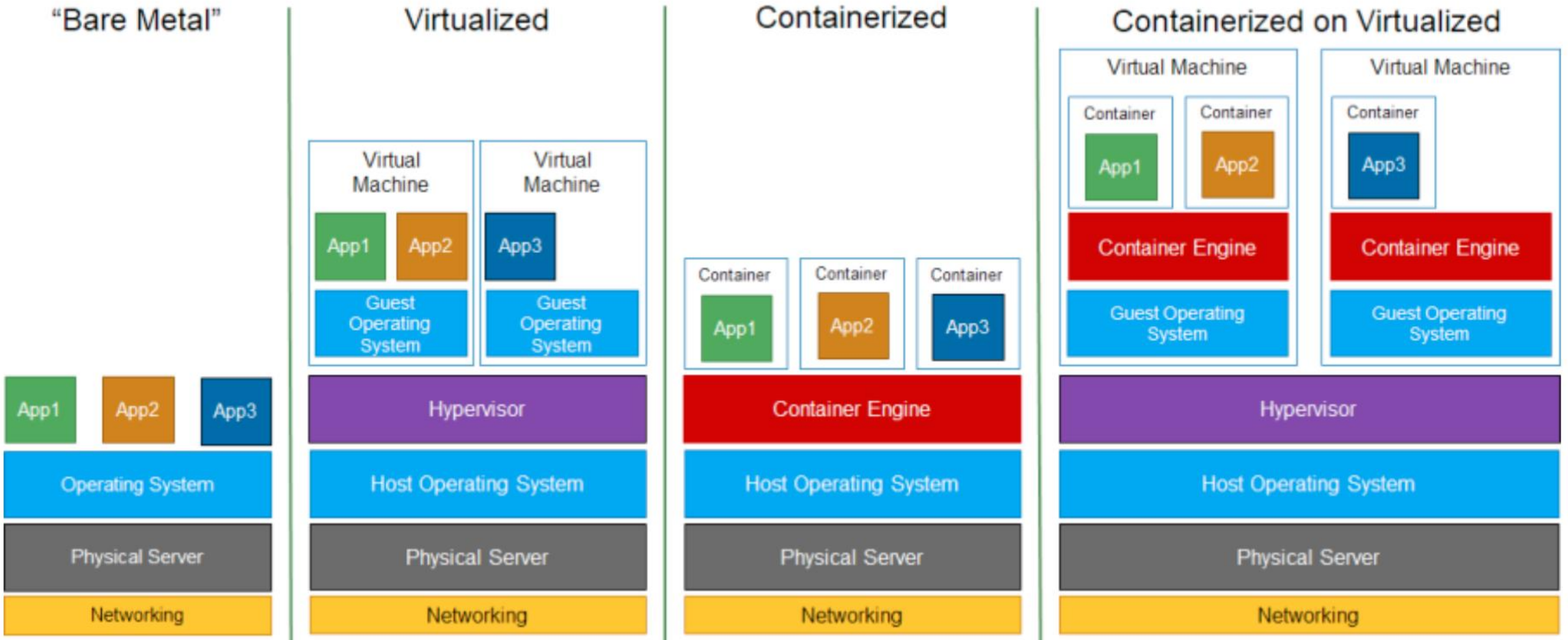
Beberapa manfaat utama dari containerization termasuk:

Portabilitas	Konsistensi	Skalabilitas	Isolasi	Efisiensi

Beberapa teknologi kontainer populer yang digunakan saat ini termasuk Docker, Kubernetes, dan containerd.

Containerization telah menjadi bagian integral dari pengembangan perangkat lunak modern dan memungkinkan organisasi untuk menghasilkan, mendistribusikan, dan mengelola aplikasi lebih cepat, konsisten, dan efisien.

Deployment mechanisms and supportability



No SQL Database

Jenis DB yang dirancang untuk mengelola, menyimpan, dan mengambil data yang tidak sesuai dengan model relasional dalam basis data SQL (Structured Query Language).

Basis data NoSQL dapat menangani jenis data yang lebih beragam dan lebih fleksibel dalam skema datanya.

Kelebihan utama dari NoSQL Database pada aplikasi meliputi:

Skalabilitas Horizontal	Kinerja Tinggi	Fleksibilitas Skema	Pemrograman yang Mudah	Kemampuan Menangani Big Data

NoSQL Database menawarkan berbagai model data, seperti dokument, grafik, key-value, dan kolom, yang dapat dipilih sesuai dengan kebutuhan aplikasi tertentu.

Web Service Specification

Web service adalah teknologi yang memungkinkan dua atau lebih aplikasi berkomunikasi dan bertukar data melalui jaringan (protokol HTTP).

Ini memungkinkan sistem yang berbeda untuk berinteraksi satu sama lain tanpa memperhatikan bahasa pemrograman atau platform yang digunakan.

Web service dapat digunakan untuk berbagai tujuan, termasuk integrasi sistem, berbagi data antar aplikasi, dan mengaktifkan interoperabilitas antar platform.

Materi yang dibahas meliputi:

OPEN API Specification	API Testing	Dan Authentication

Message Brokers dan Event Streaming

Message brokers adalah perangkat lunak atau layanan yang bertindak sebagai perantara antara aplikasi yang berkomunikasi satu sama lain dengan mengirim dan menerima pesan. Mereka menyediakan platform yang dapat diandalkan untuk mentransfer data antar aplikasi atau layanan, biasanya dalam lingkungan terdistribusi atau berbasis mikroservis.

Message broker lebih terfokus pada pengiriman pesan antara aplikasi yang berbeda, sementara event streaming lebih terfokus pada menangkap, memproses, dan merespons peristiwa dalam waktu nyata.

Model pesan dalam message broker cenderung lebih terstruktur, sementara model peristiwa dalam event streaming cenderung lebih fleksibel.

Pola komunikasi dalam message broker dapat berupa publikasi-langganan atau antrian, sementara dalam event streaming, pola komunikasi biasanya lebih terkait dengan publikasi-langganan.

Terima Kasih